SECURE SOFTWARE UPDATE MECHANISM

A Chunk-Based Update Distribution System with Edge Caching and Bitmap Resume

BCSE308P– COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Science and Engineering(Artificial Intelligence and Data Engineering) /Bachelor of Computer Science and Engineering(Cyber Security)

in

Department of Cybersecurity and Intelligent Systems

by

KAPISH P (24BDE0039)
RAJAVARSHAN M (24BYB0083)

Under the guidance of
Dr. AASEEGHA M D



School of Computer Science Engineering and Information Systems[SCORE]
VIT, Vellore

November, 2026

DECLARATION

I hereby declare that the BCSE308P - Course Project entitled “Secure Software Update Mechanism: A Chunk-Based Update Distribution System with Edge Caching and Bitmap Resume” submitted by us, for the award of the degree of *Bachelor of Computer Science and Engineering(Artificial Intelligence and Data Engineering) /Bachelor of Computer Science and Engineering(Cyber Security), School of Computer Science Engineering and Information Systems* to VIT is a record of bonafide work carried out by us under the supervision of Dr. Aaseegha M D, SCORE, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore
Date:

Kapish P (24BDE0039)
Rajavarshan M (24BYB0083)

Signature of the Candidates

CERTIFICATE

This is to certify that the **BCSE308P- Course Project** entitled “Secure Software Update Mechanism: A Chunk-Based Update Distribution System with Edge Caching and Bitmap Resume” submitted by **Kapish P (24BDE0039) and Rajavarshan M (24BYB0083)**, SCORE, VIT, for the award of the degree of *Bachelor of Computer Science and Engineering(Artificial Intelligence and Data Engineering) /Bachelor of Computer Science and Engineering(Cyber Security),School of Computer Science Engineering and Information Systems* provision during the Fall Semester 2026–27, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The Capstone project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE - Guide

ACKNOWLEDGEMENT

We would like to express our sincere thanks to our guide, Dr. Aaseegha M D, for the guidance, encouragement and valuable suggestions given to us during the reviews of this project.

We also thank the School of Computer Science Engineering and Information Systems (SCORE), VIT Vellore, for giving us the opportunity to do this course project as part of BCSE308P Computer Networks. The concepts we learnt in the theory classes, especially HTTP, TCP and the client server model, were directly used in this work.

Finally, we thank our friends and family members for their support and encouragement throughout the semester.

Kapish P
Rajavarshan M

Executive Summary

When many computers in the same lab or office download the same software update, every machine pulls the full file from the central server. This wastes WAN bandwidth and overloads the server, and a broken connection usually means starting the download again from 0%. Our Secure Software Update Mechanism, written in Python, solves these three problems together. The origin server (Flask) splits the update binary into fixed 1 MiB chunks and publishes a manifest containing the SHA-256 hash of every chunk, signed with HMAC-SHA256. An edge cache node on the local network serves chunks to clients; on a cache miss it fetches the chunk once from the origin, checks its hash and stores it. The Tkinter client keeps a bitmap (1 = received, 0 = missing) on disk, so after an interruption it requests only the missing chunks. A direct HTTP Range (206 Partial Content) mode is also supported. In our experiment with a 25 MiB update and 5 clients, the origin sent 25 MiB instead of 125 MiB, an 80% reduction, and cached clients finished in about 0.17 s instead of 8.3 s. All 25 unit tests pass, including tests for tampered manifests and corrupted chunks.

CONTENTS

Page No.

	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	vii
	List of Tables	viii
	Abbreviations	ix
	Symbols and Notations	x
1	**INTRODUCTION**	1
	1.1 Objective	1
	1.2 Motivation	1
	1.3 Background	2
2	**PROJECT DESCRIPTION AND GOALS**	3
3	**TECHNICAL SPECIFICATION**	4
4	**DESIGN APPROACH AND DETAILS**	6
	4.1 Design Approach / Materials & Methods	6
	4.2 Codes and Standards	9
	4.3 Constraints, Alternatives and Tradeoffs	9
5	**SCHEDULE, TASKS AND MILESTONES**	10
6	**PROJECT DEMONSTRATION**	11
	6.1 Sample Codes	11
	6.2 Sample Screen Shots	12
7	**RESULT & DISCUSSION**	16
8	**SUMMARY**	19
9	**REFERENCES**	20
	APPENDIX A	22

List of Figures

Figure No.	Title	Page No.
4.1	System Architecture of the Proposed Update Mechanism	6
4.2	Flowchart of the Client Download and Resume Logic	7
4.3	Sequence of a Chunked Download through the Edge Cache	8
6.1	Client Dashboard with All Cache Hits	13
6.2	First Download and Interrupt	13
6.3	Bitmap Resume and Completed Download	14
6.4	Direct HTTP Range Mode: Interrupt and Resume	14
6.5	Console Output of the Origin and Edge Servers	15
7.1	Output of the Simulation Script	16
7.2	Origin Traffic and Download Time Comparison	17
7.3	Unit Test Results	18

List of Tables

Table No.	Title	Page No.
2.1	Gaps in Existing Systems and Our Approach	3
3.1	Software and Hardware Requirements	4
3.2	HTTP Endpoints Implemented	4
3.3	HTTP Responses and Failure Handling	5
4.1	Alternatives Considered and Tradeoffs	9
5.1	Schedule, Tasks and Milestones	10
7.1	Simulation Results for 5 Clients (25 MiB Update)	16
7.2	Summary of Unit Tests	18

List of Abbreviations

API	Application Programming Interface
CDN	Content Delivery Network
GUI	Graphical User Interface
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KiB	Kibibyte (1,024 bytes)
LAN	Local Area Network
MiB	Mebibyte (1,048,576 bytes)
MOAP	Multihop Over-the-Air Programming
NAT	Network Address Translation
OS	Operating System
P2P	Peer-to-Peer
RFC	Request for Comments
SHA-256	Secure Hash Algorithm 256-bit
TCP	Transmission Control Protocol
TUF	The Update Framework
URL	Uniform Resource Locator
WAN	Wide Area Network
WSN	Wireless Sensor Network

Symbols and Notations

S	Total size of the update file in bytes
C	Fixed chunk size (1 MiB = 1,048,576 bytes)
N	Number of chunks in the update
i	Chunk index (0 to N − 1)
b_i	Bitmap bit of chunk i (1 = received, 0 = missing)
H(x)	SHA-256 hash of data x
K	Shared signing key
σ	HMAC-SHA256 signature of the manifest
M	Manifest (file metadata and list of chunk hashes)
B_origin	Bytes sent by the origin server
B_delivered	Bytes delivered to all clients
k	Number of clients
⌈x⌉	Ceiling of x (smallest integer ≥ x)

CHAPTER 1
INTRODUCTION

A software update is one of the most common file transfers that happens on any computer network. Operating systems, browsers, antivirus programs and even small tools regularly download new versions of themselves. In most systems this is done as a simple client server request: each machine connects to the vendor server and downloads the complete update file over HTTP, which runs on top of TCP. This works for a single computer, but it does not scale well when hundreds of machines in the same building ask for the same file at the same time.

From a Computer Networks point of view, the update problem touches many topics we studied in the course: the application layer protocol (HTTP), reliable byte stream transport (TCP), caching near the client, partial transfers, and integrity of data received over an untrusted network. Our project brings these together in a working system written in Python, with an origin server, an edge cache node and a desktop client.

1.1 Objective

The main objective of this project is to design and implement a network-efficient and secure software update mechanism that:

- ✓ splits the update binary into fixed 1 MiB chunks and describes them in a signed manifest,
- ✓ serves repeated requests from a local edge cache so that each chunk crosses the WAN only once,
- ✓ tracks received chunks with a bitmap so that an interrupted download resumes from where it stopped, and
- ✓ verifies every chunk and the final file with SHA-256 before the update is accepted.

1.2 Motivation

Consider a computer lab where every system downloads a large update separately. If 60 machines each download a 500 MB update, about 30 GB moves over the external link even though the content is the same. During major releases the vendor servers also become slow because of thousands of parallel downloads.

The second problem is unreliable connectivity. Wi-Fi drops, server restarts or a laptop going to sleep can break a long download. Many simple downloaders then start again from 0%, which wastes time and bandwidth again. The third problem is trust: if a file is fetched from a cache or another intermediate node, the client must be able to check that the bytes were not changed on the way. These three problems motivated us to build a system that handles bandwidth, resumption and integrity together instead of only one of them.

1.3 Background

Software update distribution has been studied using centralized CDNs, peer-to-peer swarms and multicast protocols. Gkantsidis et al. [1] analysed planet-scale update traffic at Microsoft and showed that central servers face huge load peaks during release windows, which motivates caching and peer assistance. Updaticator [2] proposed distributing updates through untrusted cache-enabled networks while the client still verifies the update, which is close to our idea of an edge node that is not trusted blindly.

BitTorrent style swarms, measured by Izal et al. [3], reduce server load by letting peers exchange pieces, but they bring peer churn, unpredictable speeds and NAT traversal problems that are difficult to manage in an enterprise network. Code dissemination protocols for wireless sensor networks such as Deluge [4] and MOAP [5] introduced the idea of dividing an image into pages and keeping a bit vector of received pages, which inspired our bitmap. However, they target low-power radios and not high-speed LAN and WAN links. Multicast based distribution [6], [7], [8] reduces traffic by sending one stream to many receivers, but reliable multicast across the Internet is still hard because of loss recovery and receivers with different speeds. Work on optimizing file transfers in high-speed networks [9] and decentralized patch delivery for IoT [10] shows that this area is still active. For update security, The Update Framework (TUF) [11] showed that signed metadata is necessary when files come through mirrors.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

Our system has three programs that talk to each other over HTTP. The **origin server** keeps the original update file and creates a manifest for it. The **edge cache node** runs on the local network and acts as a middle layer between the clients and the origin. The **client** is a Tkinter desktop dashboard that downloads the update and shows everything that happens on the network in real time: the online status of both servers, a colour-coded map of the 25 chunks (cache HIT, cache MISS, downloading or missing), a live throughput graph, the edge cache statistics and an event log. The user can interrupt and resume the download, clear the edge cache and reset the local files from the same window. For comparison, the client also has a direct mode in which it downloads the whole file from the origin using HTTP Range requests.

In the first review we identified four main gaps in the existing approaches. Table 2.1 lists them along with the part of our system that addresses each gap.

Table 2.1 Gaps in Existing Systems and Our Approach

Gap in existing systems	Our approach
Centralized server bottleneck: every client downloads from the origin, which saturates it during releases	Edge cache on the LAN; each chunk is fetched from the origin only once
Redundant subnet fetches: normal proxies have no chunk-level tracking	Fixed 1 MiB chunks with an index and SHA-256 per chunk, cached by file hash
Full transfer restarts after a connection drop	Client bitmap saved on disk; only chunks with bit 0 are requested again
P2P overhead: peer churn, slow seeds, no delivery guarantee	Deterministic hierarchy (origin to edge to client) with integrity checks at every hop

The goals that we set for the project are given below.

1. Implement an origin server that publishes a signed chunk manifest and serves individual chunks.
2. Implement an edge cache that returns HIT or MISS and never stores a chunk whose hash is wrong.
3. Implement a client with bitmap-based resume and a direct HTTP Range resume mode.
4. Measure the origin traffic with and without the edge cache and test all the failure cases.

CHAPTER 3
TECHNICAL SPECIFICATION

The whole project is written in Python. Only two third-party packages are used (Flask and Requests); hashing, HMAC, threads, the GUI and the tests use the Python standard library. Table 3.1 gives the software and hardware requirements, Table 3.2 lists the HTTP endpoints of the two servers and Table 3.3 shows how errors are handled.

Table 3.1 Software and Hardware Requirements

Item	Specification
Language	Python 3.10 or later (developed on Python 3.13, also tested on Python 3.12)
Server framework	Flask 3.1.0 (development server, threaded mode)
HTTP client	Requests 2.32.3
GUI	Tkinter / ttk (included with the standard Python installer)
Security	hashlib (SHA-256), hmac (HMAC-SHA256)
Testing	unittest (25 test cases)
Operating system	Windows 10/11 or Linux
Hardware	Any PC or laptop with about 100 MB free disk space; one machine is enough for the demo (ports 5000 and 5001 on 127.0.0.1)
Sample update	sample_update.bin, 26,214,400 bytes (25 MiB), created automatically
Chunk size	1 MiB (1,048,576 bytes), so the sample file has 25 chunks

Table 3.2 HTTP Endpoints Implemented

Program	Endpoint	Purpose
Origin (5000)	GET /manifest	Signed manifest with all chunk hashes
Origin (5000)	GET /chunk/<file>/<i>	Returns chunk i (1 MiB)
Origin (5000)	GET /download/<file>	Full file (200) or byte range (206 / 416)
Origin (5000)	GET /metadata	Version, size and SHA-256 of the file
Edge (5001)	GET /manifest	Relays and verifies the origin manifest
Edge (5001)	GET /chunk/<file>/<i>	Chunk from cache (HIT) or origin (MISS)
Edge (5001)	POST /cache/clear	Deletes cached chunks and resets counters
Both	GET /stats, /health	Traffic counters and health check

The system also has to behave correctly when something goes wrong in the network. Table 3.3 lists the main HTTP responses and failure cases and how each part of the system handles them.

Table 3.3 HTTP Responses and Failure Handling

Situation	Response / Detection	Action taken
Normal full download (direct mode)	200 OK	Saved to .part, size and SHA-256 checked
Resume with Range: bytes=N-	206 Partial Content	Content-Range and Content-Length checked before appending
Range outside the file	416 Range Not Satisfiable	Reported to user, .part file not modified
Server returns 200 when 206 was expected	Status check in client	Response rejected, never appended to .part
Filename with ../ or folder path	400 Bad Request	Request refused (path traversal protection)
Chunk index out of range	404 Not Found	Error shown to user
Origin not reachable from edge	502 Bad Gateway	Client keeps bitmap and can resume later
Tampered manifest	HMAC mismatch	Manifest rejected, no download starts
Corrupted chunk	SHA-256 mismatch	Edge: not stored; client: retried, bit stays 0
Connection drop / server stopped	requests exception	Partial data and bitmap kept on disk

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 Design Approach / Materials & Methods

We followed a hybrid design: the origin server is the only authority for the update content, while the edge cache reduces repeated traffic on the WAN. Fig. 4.1 shows the overall architecture. The clients normally talk only to the edge node over the LAN. The edge node talks to the origin only when it does not already have a chunk.

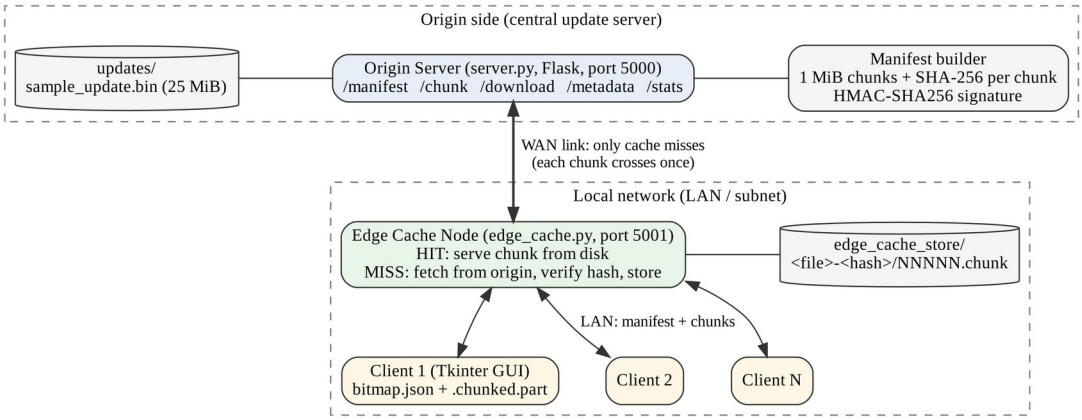


Fig. 4.1 System Architecture of the Proposed Update Mechanism

Our methodology is divided into five operational segments, which are the same five points we presented in the first review.

1. **Binary partitioning.** The update of size S is divided into fixed chunks of size C = 1 MiB. The number of chunks N and the position of chunk i are given by Eqn. 4.1 and Eqn. 4.2.

$$N = \lceil S / C \rceil \tag{4.1}$$

$$offset_i = i \times C, \quad \text{length}_i = \min(C, S - i \times C) \tag{4.2}$$

For the sample file S = 26,214,400 bytes, so Eqn. 4.1 gives N = 25 chunks of exactly 1 MiB each. The last chunk can be shorter for other file sizes, which is why the length in Eqn. 4.2 uses min().

2. **Manifest distribution.** The origin builds a manifest M containing the file name, version, size, the whole-file SHA-256 and, for every chunk, its index, offset, length and SHA-256. The manifest is converted to canonical JSON (sorted keys, no extra spaces) and signed with a shared key K as in Eqn. 4.3. Any change to even one hash makes the signature check fail.

$$\sigma = \text{HMAC-SHA256}(K, \text{canonical}(M)) \tag{4.3}$$

3. **Edge cache interception.** For each chunk request the edge first looks in its store. If a valid copy exists it is a HIT. Otherwise it is a MISS: the edge fetches the chunk from the origin, compares its SHA-256 with the manifest and only then stores and returns it. Cached chunks are kept in a folder named after the file hash, so a new version never reuses chunks of an old version. A lock per chunk makes sure that two clients missing the same chunk at the same time cause only one origin fetch.
4. **Bitmap state tracking.** The client keeps a bit b_i for every chunk (1 = received, 0 = missing) in a small JSON file next to the partial download. A bit is set to 1 only after the chunk passes its SHA-256 check and is written at its offset.
5. **Resumable session recovery.** After an interruption or a network error the bitmap and the partial file stay on disk. On resume, the client verifies the manifest again and requests only the chunks with $b_i = 0$. When all bits are 1, the full file is checked against the whole-file SHA-256 and only then renamed to its final name.

The decision logic of the client for one download is shown as a flowchart in Fig. 4.2. The same loop is used for Start and for Resume; the only difference is whether the bitmap begins with all zeros or is loaded from disk.

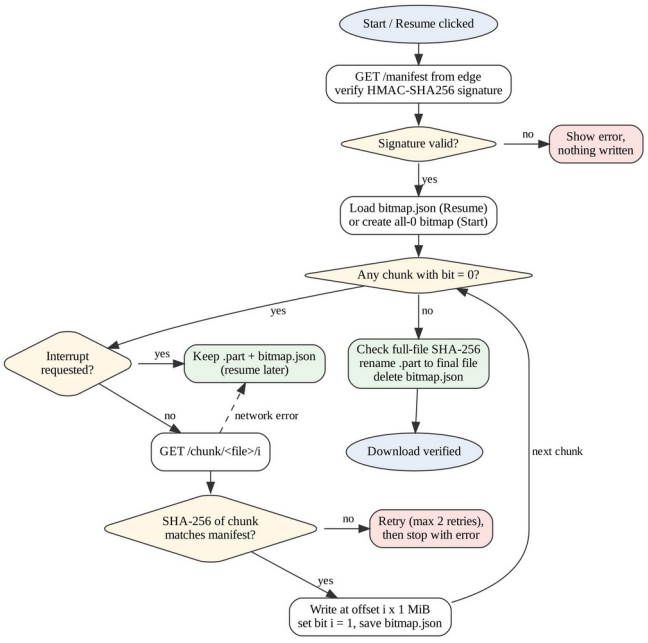


Fig. 4.2 Flowchart of the Client Download and Resume Logic

A part of the real manifest produced by the origin for the 25 MiB sample file is shown below (only the first chunk entry is printed). Every client and the edge node recompute the HMAC over the manifest fields and compare it with the signature field before trusting any hash.

```
{
  "filename": "sample_update.bin",
  "version": "1.0.0",
  "size": 26214400,
  "sha256": "25fec3bde0fb571042ece4eb835ab77216c93d83732fc4f064db0318ba59b15b",
  "chunk_size": 1048576,
  "chunk_count": 25,
  "chunks": [
    {"index": 0, "offset": 0, "length": 1048576,
     "sha256": "0d467c555a791f840d5cb7992efa681b37180a0db1e4539875c789ef55b74a4c"},
    ...  (24 more entries, the last one has offset 25165824)
  ],
  "signature": "5e03cb012f612a73d7d067b042de898f2a39a08c89297a077dea45a660053369"
}
```

The bitmap state file written by the client after the download was interrupted at 11 chunks looks as follows. Because the file hash is stored in it, a bitmap saved for an older version of the update is rejected and cannot be mixed with chunks of a new version.

```
{
  "filename": "sample_update.bin",
  "version": "1.0.0",
  "file_sha256": "25fec3bde0fb571042ece4eb835ab77216c93d83732fc4f064db0318ba59b15b",
  "chunk_size": 1048576,
  "chunk_count": 25,
  "bitmap": "11111111111000000000000000"
}
```

Fig. 4.3 shows the message sequence for one chunked download through the edge cache.

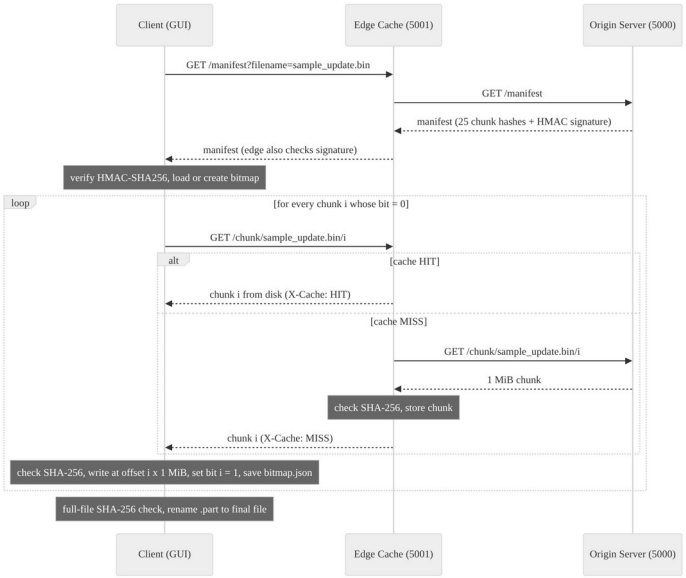


Fig. 4.3 Sequence of a Chunked Download through the Edge Cache

4.2 Codes and Standards

The implementation follows these standards:

- ✓ **RFC 9110 (HTTP Semantics)** [12] for request methods, status codes 200, 206 and 416, and the Range, Content-Range and Content-Length headers used in the direct resume mode.
- ✓ **RFC 9293 (TCP)** [15]: HTTP runs over TCP, which gives a reliable ordered byte stream. Our 64 KiB and 1 MiB pieces are application-level chunks, not TCP segments.
- ✓ **FIPS 180-4** [14] for SHA-256, used for every chunk and for the whole file.
- ✓ **RFC 2104** [13] for HMAC, used to sign the manifest (HMAC-SHA256).
- ✓ **PEP 8** coding style for Python and the Python unittest framework for testing.

4.3 Constraints, Alternatives and Tradeoffs

The main constraints were that the project had to run on a normal student laptop, use only free tools, and be demonstrable without a real multi-site network. So the origin, edge and clients run as separate processes on one machine, and a small delay of 0.02 s per 64 KiB piece at the origin is used to emulate a slow WAN link (about 3.1 MiB/s). Table 4.1 lists the main design alternatives that we considered.

Table 4.1 Alternatives Considered and Tradeoffs

Decision	Alternative	Reason for our choice / tradeoff
Edge cache hierarchy	P2P swarm (BitTorrent)	Predictable and easy to manage in a LAN; but the edge node is a single point that must be deployed
Fixed 1 MiB chunks	Smaller (64 KiB) or larger (8 MiB) chunks	25 requests for 25 MiB keeps overhead low and a lost connection wastes at most 1 MiB
HMAC-SHA256 signature	Public-key signature (Ed25519, RSA)	No extra library needed; but the key is shared, so a real product should use public-key signatures
HTTP over TCP (unicast)	Reliable multicast	Works through normal routers and firewalls; multicast would save LAN traffic but is hard to deploy
Flask development server	Gunicorn / Nginx	Simple for a lab demo; not meant for production load

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The project was done by a team of two members. The work was divided week by week as shown in Table 5.1. Both members took part in testing and in preparing the reviews.

Table 5.1 Schedule, Tasks and Milestones

Week	Task	Milestone / Output
1 – 2	Problem study, literature review of 10 papers, gap analysis	Problem statement and gaps
3	Abstract, methodology and presentation	Review 1 (PPT presented)
4 – 5	Flask origin server, /metadata and /download with Range support	Working full and partial download
6	Tkinter client, .part file, interrupt and resume (HTTP 206)	Direct resume mode working
7	Unit tests for ranges, 416, corruption and connection failure	16 tests passing
8	1 MiB chunk manifest with HMAC-SHA256 signature	/manifest and /chunk endpoints
9	Edge cache node with HIT / MISS and hash checking	edge_cache.py
10	Bitmap state file, chunked resume, dashboard GUI (chunk map, speed graph, event log)	Chunked mode and dashboard
11	Simulation script, more unit tests, measurements	25 tests passing, results
12	Report writing and final demonstration	Final review

CHAPTER 6
PROJECT DEMONSTRATION

The project folder contains server.py (origin), edge_cache.py (edge node), client.py (GUI client), update_manifest.py (manifest helpers), simulate_clients.py (experiment) and tests.py (unit tests). For the demonstration, the origin and edge are started in two terminals and the client in a third, as explained in Appendix A.

6.1 Sample Codes

Three important parts of the code are shown here. The first function (update_manifest.py) builds the chunk table. It reads the file only once and computes the hash of each chunk and of the whole file at the same time.

```
def build_manifest(file_path: Path, version: str, chunk_size: int = CHUNK_SIZE) -> dict:
    """Read the file once and record offset, length and SHA-256 for every chunk."""
    total_size = file_path.stat().st_size
    whole_file_hash = hashlib.sha256()
    chunks = []
    with file_path.open("rb") as source_file:
        index = 0
        while True:
            data = source_file.read(chunk_size)
            if not data:
                break
            whole_file_hash.update(data)
            chunks.append(
                {
                    "index": index,
                    "offset": index * chunk_size,
                    "length": len(data),
                    "sha256": hashlib.sha256(data).hexdigest(),
                }
            )
            index += 1
    ...   # body = {filename, version, size, sha256, chunk_size, chunk_count, chunks}
body["signature"] = sign_manifest(body)
return body
```

The second part is the core of the edge cache (edge_cache.py). A chunk is stored only after its SHA-256 matches the signed manifest.

```
with lock_for(str(path)):
    data = read_cached_chunk(path, entry)
    if data is not None:
        cache_status = "HIT"
        add_stat("hits")
    else:
        cache_status = "MISS"
        try:
            data = fetch_chunk_from_origin(filename, index)
        except requests.RequestException as exc:
            return jsonify({"error": f"Origin server not reachable: {exc}"}), 502
        if not chunk_hash_matches(data, entry):
            return jsonify({"error": f"Chunk {index} from origin failed the SHA-256 check."}), 502
        store_chunk(path, data)
        add_stat("misses")
        add_stat("bytes_from_origin", len(data))
```

The third part is the client loop (client.py), which requests only the missing chunks. The bitmap is saved after every chunk, so an interruption at any point loses at most the chunk that was in transfer.

```
for index in missing_chunks(bitmap):
    if stop_event is not None and stop_event.is_set():
        raise DownloadInterruptedError("The user interrupted the download.")
    entry = manifest["chunks"][index]

    for attempt in range(MAX_CHUNK_RETRIES + 1):
        data, cache_status = fetch_chunk(server_url, manifest["filename"], index)
        if chunk_hash_matches(data, entry):
            break
        summary["retries"] += 1
        if attempt == MAX_CHUNK_RETRIES:
            raise ChunkIntegrityError(f"Chunk {index} failed the SHA-256 check {attempt + 1}
times.")

    write_chunk(part_file, entry["offset"], data)
    bitmap[index] = 1
    save_bitmap_state(state_file, manifest, bitmap)
```

6.2 Sample Screen Shots

Fig. 6.1 shows the complete client dashboard after two downloads of the update. The window is divided into cards. On the left are the update source and transfer mode, the progress bar with speed, the chunk map and the throughput graph. On the right are the integrity panel (manifest signature and full-file SHA-256), the HIT and MISS count for the current download, the live edge cache statistics read from the /stats endpoint, and a time-stamped event log. The two dots in the header show whether the origin (port 5000) and the edge (port 5001) are online; the client checks their /health endpoints every 1.5 s. In this screenshot every chunk came from the edge cache (25 HIT, 0 MISS), so all boxes are cyan. The edge statistics show 25.0 MB fetched from the origin and 50.0 MB served to clients, which is a 50% origin offload for two downloads.

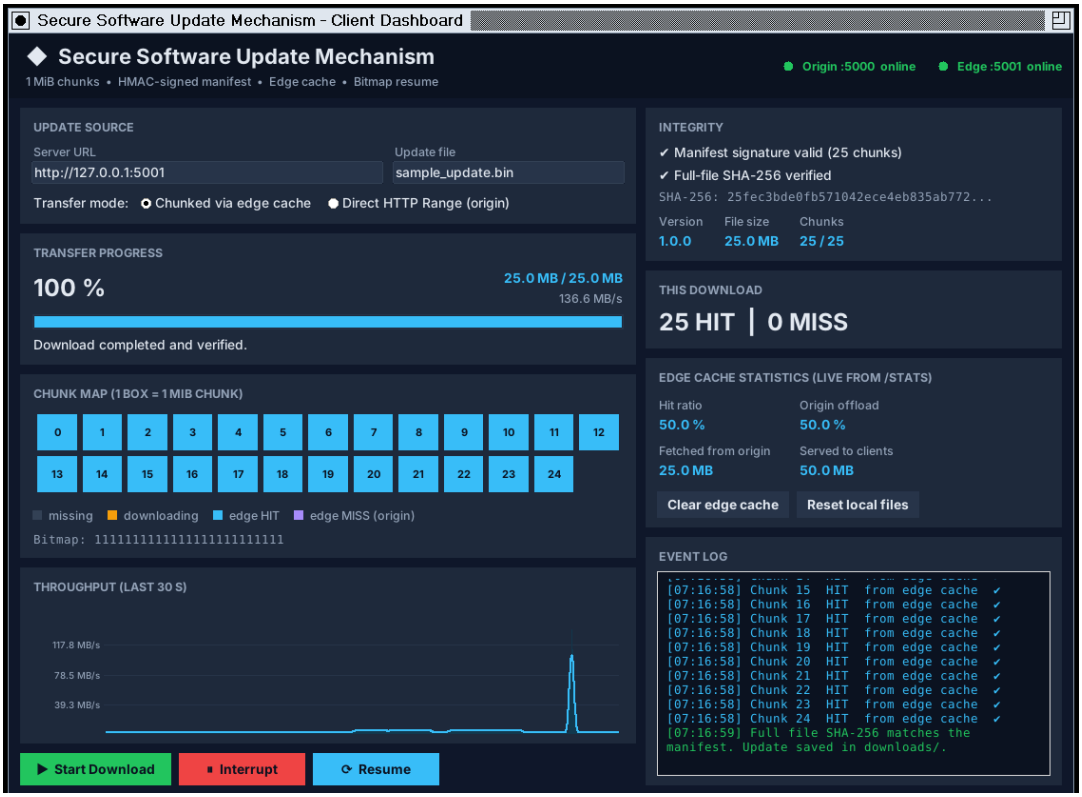


Fig. 6.1 Client Dashboard with All Cache Hits

Fig. 6.2 shows the first download with an empty cache. On the left, 9 chunks have arrived (violet boxes, all cache MISS) and chunk 9 is being downloaded (amber box). We then clicked Interrupt (right): the client kept 11 of 25 chunks, saved the bitmap 11111111111000000000000000 in sample_update.bin.bitmap.json, and the event log recorded the interruption.

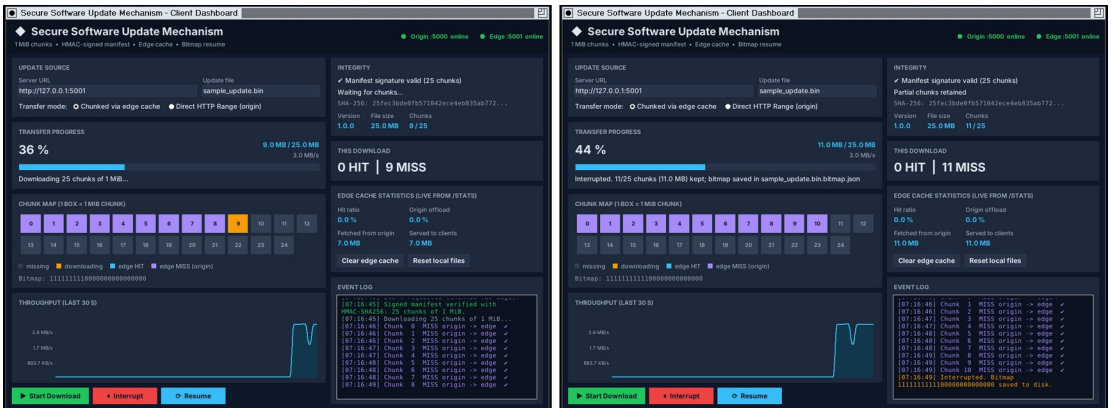


Fig. 6.2 First Download and Interrupt

After clicking Resume, the client verified the manifest again and requested only the 14 missing chunks, starting from chunk 11 (Fig. 6.3, left). The right side of Fig. 6.3 shows the end of this session: all 25 boxes are filled, the progress is 100%

and the full-file SHA-256 was verified. The gap in the throughput graph is the time when the download was interrupted.

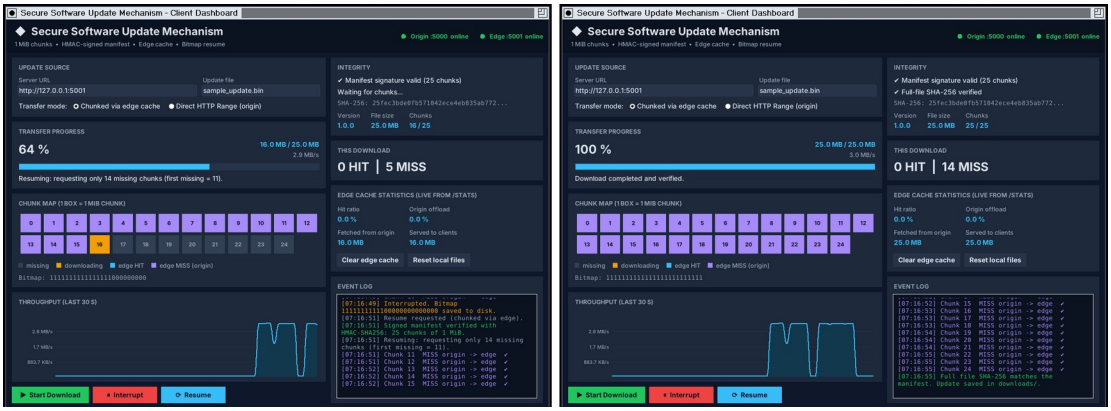


Fig. 6.3 Bitmap Resume and Completed Download

The direct mode was also tested (Fig. 6.4). Here the chunk map is not used. The transfer was interrupted after 9,568,256 bytes; on resume the client sent Range: bytes=9568256- and the server answered with 206 Partial Content for the remaining 16,646,144 bytes, which add up to the full 26,214,400 bytes. The event log shows the 200 OK of the first request and the 206 of the resumed request.

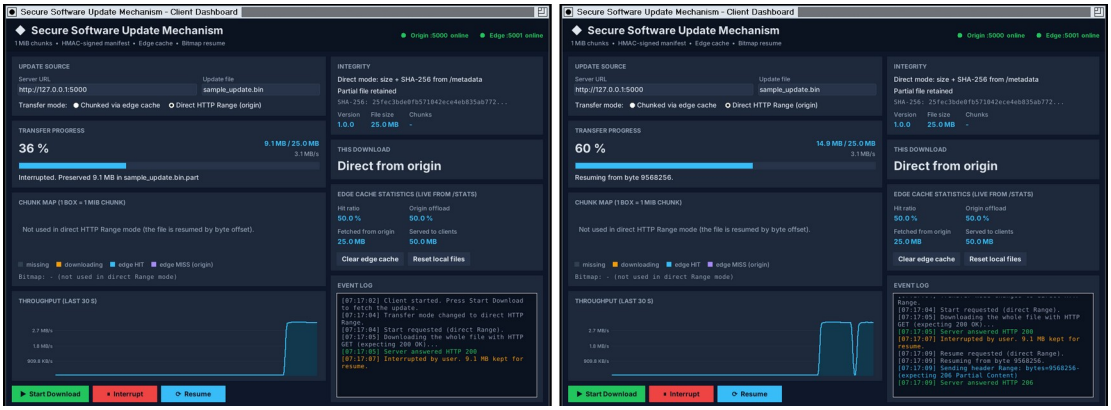


Fig. 6.4 Direct HTTP Range Mode: Interrupt and Resume

Fig. 6.5 shows the console output of both servers for the same session. Each chunk was requested from the origin only once, and the edge log shows MISS for the first download and HIT for the second one.

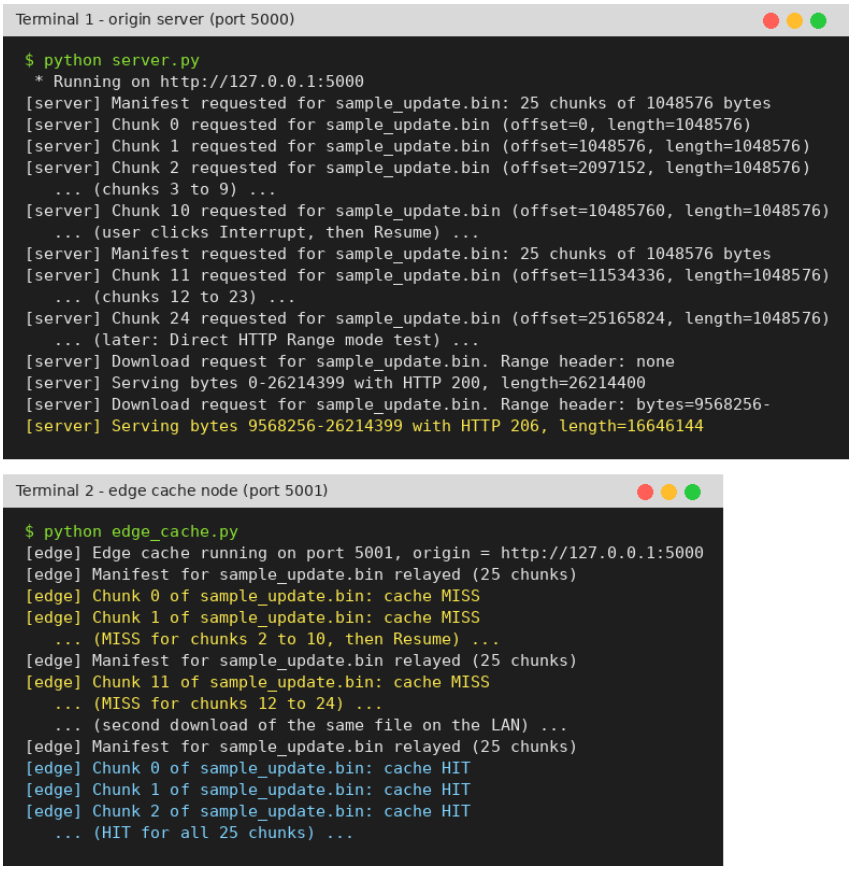


Fig. 6.5 Console Output of the Origin and Edge Servers

CHAPTER 7
RESULT & DISCUSSION

To measure the benefit of the edge cache we wrote simulate_clients.py. It starts the origin (port 5000) and the edge (port 5001) and runs 5 headless clients that use the same download functions as the GUI. Three scenarios were measured with the 25 MiB sample file: all clients going directly to the origin, clients going through the edge one after another, and all clients starting together through an empty edge cache. A fourth scenario interrupts a download after 10 chunks and resumes it. The console output of the script is shown in Fig. 7.1 and the results are summarised in Table 7.1.

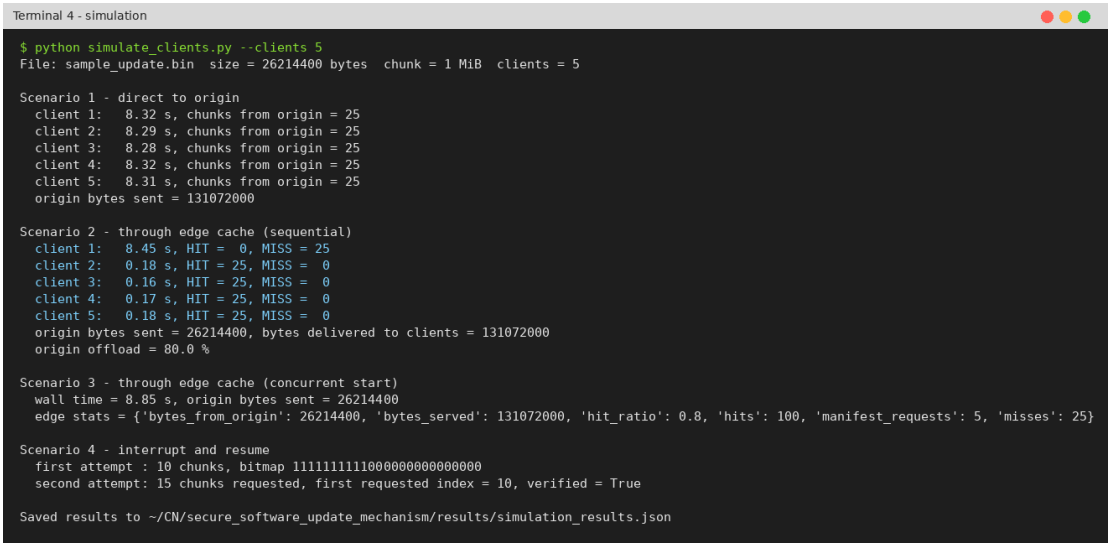


Fig. 7.1 Output of the Simulation Script

Table 7.1 Simulation Results for 5 Clients (25 MiB Update)

Scenario	Origin bytes sent	Cache HIT / MISS	Time per client
Direct to origin	131,072,000 (125 MiB)	–	8.28 – 8.32 s
Edge, sequential	26,214,400 (25 MiB)	100 / 25	8.45 s (1st), 0.16 – 0.18 s (others)
Edge, all together	26,214,400 (25 MiB)	100 / 25	8.85 s wall time for all 5
Interrupt + resume	–	–	2nd attempt requested 15 of 25 chunks

The origin offload is calculated using Eqn. 7.1, where B_origin is the number of chunk bytes sent by the origin and B_delivered is the number of bytes delivered to all k clients.

$$\text{Offload (\%)} = (1 - B_{origin} / B_{delivered}) \times 100 \qquad (7.1)$$

With k = 5 clients, B_delivered = 5 × 26,214,400 = 131,072,000 bytes and B_origin = 26,214,400 bytes, so the offload is (1 − 0.2) × 100 = 80%. In general the offload is (1 − 1/k) × 100 when the cache starts empty, so it grows with the number of clients. This matches the “up to 80%” offload that we expected in Review 1. Fig. 7.2 compares the origin traffic and the time per client.

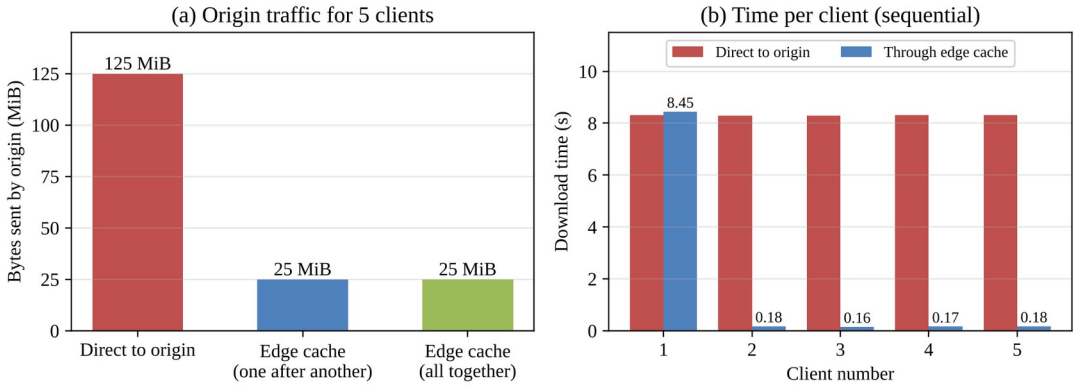


Fig. 7.2 Origin Traffic and Download Time Comparison

Some observations from the results:

- ✓ Even when all 5 clients started at the same moment, the origin sent each chunk only once, because the per-chunk lock made the other requests wait for the first fetch and then served them as HITs.
- ✓ The first client through the edge takes slightly longer than a direct client (8.45 s vs about 8.3 s) because of the extra hop and hash check. Every later client is about 49 times faster (0.17 s vs 8.3 s) since the chunks come from the local cache.
- ✓ In the resume scenario only 15 of 25 chunks (15 MiB instead of 25 MiB) were requested again, so 40% of the transfer was saved, and the final SHA-256 check passed.
- ✓ Table 7.2 summarises the 25 unit tests and Fig. 7.3 shows their output. All of them passed in 0.171 s.

Table 7.2 Summary of Unit Tests

Test group	No. of tests	What is checked
Server routes	7	health, metadata, 200 / 206 / 416 responses, range parser, path traversal
Range resume client	9	resume offset, wrong offset, 200 never appended, Content-Range, size and hash checks, connection failure
Manifest and bitmap	6	chunk hashes, tampered manifest rejected, bitmap resume, corrupted chunk, old bitmap rejected
Edge cache	3	MISS then HIT, damaged cache file fetched again, cache clear and counter reset

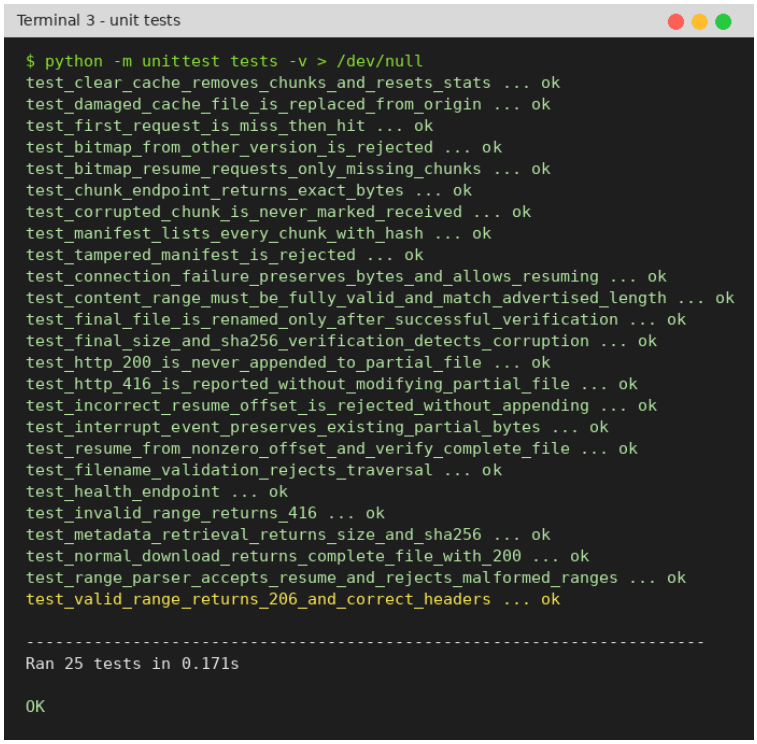


Fig. 7.3 Unit Test Results

A limitation is that the time values were measured on one machine with an emulated WAN delay, so the edge (LAN) times are faster than they would be on a real network. The byte counts and the 80% offload, however, do not depend on the machine, because they come from the number of chunks actually sent by the origin.

CHAPTER 8
SUMMARY

In this project we built a Secure Software Update Mechanism that combines a central origin server, a local edge cache and a resumable client. The update is split into 1 MiB chunks described by an HMAC-SHA256 signed manifest. The edge cache fetches each chunk from the origin only once and checks its hash before storing it, and the client uses a bitmap to resume interrupted downloads by requesting only the missing chunks. A direct HTTP Range mode (206 Partial Content) is also provided.

Our measurements with a 25 MiB update and 5 clients showed that the origin traffic dropped from 125 MiB to 25 MiB, which is an 80% reduction, and that clients served from the cache finished in about 0.17 s instead of 8.3 s. An interrupted download resumed by requesting 15 of the 25 chunks. All 25 unit tests, including tests for tampered manifests and corrupted chunks, passed.

For future work we recommend the following:

- ✓ Replace the shared HMAC key with a public-key signature (for example Ed25519) so that clients only need the public key, following the ideas of TUF [11].
- ✓ Download several missing chunks in parallel and add HTTPS (TLS) between all nodes.
- ✓ Support more than one edge node and let clients choose the nearest one automatically.
- ✓ Add an eviction policy (such as LRU) and a size limit for the edge cache store.

CHAPTER 9
REFERENCES

[1] C. Gkantsidis, T. Karagiannis, P. Rodriguez, and M. Vojnović, “Planet scale software updates,” in Proc. ACM SIGCOMM, Sep. 2006.

[2] M. Ambrosin, C. Busold, M. Conti, A.-R. Sadeghi, and M. Schunter, “Updaticator: Updating billions of devices by an efficient, scalable and secure software update distribution over untrusted cache-enabled networks,” in Computer Security – ESORICS 2014, LNCS, vol. 8712, Springer, 2014, doi: 10.1007/978-3-319-11203-9_5.

[3] M. Izal, G. Urvoy-Keller, E. W. Biersack, P. A. Felber, A. Al Hamra, and L. Garcés-Erice, “Dissecting BitTorrent: Five months in a torrent’s lifetime,” in Proc. Passive and Active Measurement Workshop (PAM 2004), LNCS, vol. 3015, Springer, 2004.

[4] J. W. Hui and D. Culler, “The dynamic behavior of a data dissemination protocol for network programming at scale,” in Proc. 2nd ACM Conf. Embedded Networked Sensor Systems (SenSys), 2004, pp. 81–94.

[5] T. Stathopoulos, J. Heidemann, and D. Estrin, “A remote code update mechanism for wireless sensor networks,” Technical Report CENS-TR-30, University of California, Los Angeles, Nov. 2003.

[6] M. Gumbold, “Software distribution by reliable multicast,” in Proc. 21st IEEE Conf. Local Computer Networks (LCN), 1996, doi: 10.1109/LCN.1996.558150.

[7] M. Handley, S. Floyd, B. Whetten, R. Kermode, L. Vicisano, and M. Luby, “The reliable multicast design space for bulk data transfer,” RFC 2887, IETF, Aug. 2000.

[8] S. Shi and M. Waldvogel, “A rate-based end-to-end multicast congestion control protocol,” in Proc. 5th IEEE Symp. Computers and Communications (ISCC), 2000, pp. 678–686.

[9] M. Arifuzzaman and E. Arslan, “Online optimization of file transfers in high-speed networks,” in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC21), 2021.

[10] J. Lee, “Patch Transporter: Incentivized, decentralized software patch system for WSN and IoT environments,” Sensors, vol. 18, no. 2, Art. no. 574, 2018, doi: 10.3390/s18020574.

[11] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, “Survivable key compromise in software update systems,” in Proc. 17th ACM Conf. Computer and Communications Security (CCS), 2010.

[12] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, IETF, Jun. 2022.

[13] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, IETF, Feb. 1997.

[14] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[15] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, IETF, Aug. 2022.

[16] Pallets Projects, “Flask documentation (version 3.1.x).” [Online]. Available: https://flask.palletsprojects.com/

APPENDIX A

Steps to Run the Project

Open a terminal in the project folder (secure_software_update_mechanism) and install the two dependencies once:

```
python -m pip install -r requirements.txt
```

Start each program in its own terminal and keep them open:

```
python server.py          # Terminal 1: origin server on http://127.0.0.1:5000
python edge_cache.py      # Terminal 2: edge cache on http://127.0.0.1:5001
python client.py          # Terminal 3: GUI client
```

In the client, keep the transfer mode as “Chunked via edge cache” and click Start Download. Click Interrupt while the chunk map is filling and then Resume; only the grey (missing) chunks are requested. Click Start Download again to see a download with all cache hits. The Clear edge cache button empties the edge cache (POST /cache/clear) so that the demonstration can be repeated with a cold cache, and Reset local files deletes the partial and completed downloads. Select “Direct HTTP Range (origin)” to test the 206 Partial Content resume mode. The experiment and the tests are run as follows (stop Terminals 1 and 2 before running the simulation, because it uses the same ports):

```
python simulate_clients.py --clients 5     # results saved in results/simulation_results.json
python -m unittest tests -v
```

Files created at run time: downloads/sample_update.bin.chunked.part and downloads/sample_update.bin.bitmap.json (chunked mode, removed after success), downloads/sample_update.bin.part (direct mode), and edge_cache_store/ (edge cache). The signing key can be changed with the environment variable UPDATE_SIGNING_KEY; the same value must be used by all three programs.